AI-Based Medical Report Summarization Using NLP

Oct 28, 2026 · @hh

1. Project Title

AI-Based Medical Report Summarization Using NLP

A mini project in Artificial Intelligence and Natural Language Processing, applied to healthcare.

2. Abstract

This project builds a web application that turns long medical reports into short, accurate summaries using Natural Language Processing (NLP). Users upload a discharge summary, lab report or radiology report as text, PDF or scanned image. The system extracts the text, cleans it, and identifies key medical entities such as diseases, medicines, test values and procedures.

It then produces two outputs: a concise clinical summary for doctors and a plain-language summary for patients. Summarization uses both extractive methods (TextRank) and abstractive transformer models (BART / T5, fine-tuned on medical text). Quality is measured with ROUGE and BERTScore, along with a manual check for factual accuracy. The goal is to save clinicians reading time and help patients understand their own reports.

3. Problem Statement

Medical reports are long, full of jargon, and written in inconsistent formats, so finding the key facts takes time. Doctors reviewing a patient's history must read many pages of discharge notes, lab results and imaging reports. Patients usually cannot understand terms such as "bilateral pleural effusion" or abnormal lab values without help.

The main issues are:

- **Information overload:** clinicians spend a large share of their time reading and writing documentation instead of treating patients.
- **Low health literacy:** patients misread or ignore important findings in their own reports.
- **Unstructured data:** reports arrive as free text, PDFs or scanned images, which are hard to search or compare.

- **Risk of missed details:** under time pressure, a critical value or diagnosis buried in a long report can be overlooked.

There is a need for an automated tool that reads a medical report and produces a short, faithful summary for both doctors and patients.

4. Objectives

1. Accept medical reports as plain text, PDF or scanned image, and extract clean text using parsing and OCR.
2. Pre-process clinical text: expand common abbreviations, remove noise, and split it into sections (history, findings, impression, medication).
3. Identify medical entities (diseases, symptoms, drugs, dosages, lab tests and values) using Named Entity Recognition.
4. Generate an extractive summary (TextRank) and an abstractive summary (fine-tuned BART / T5) and compare them.
5. Produce two summary styles: a technical one for doctors and a simple-language one for patients.
6. Highlight abnormal lab values and critical findings in the output.
7. Evaluate summaries with ROUGE-1, ROUGE-2, ROUGE-L and BERTScore, plus a manual factual-accuracy check.
8. Deliver everything through a simple, easy-to-use web interface.

5. Existing System

Today, most medical reports are read and summarized by hand. A doctor reads the full report, and a patient either asks the doctor or searches the internet for terms they do not understand.

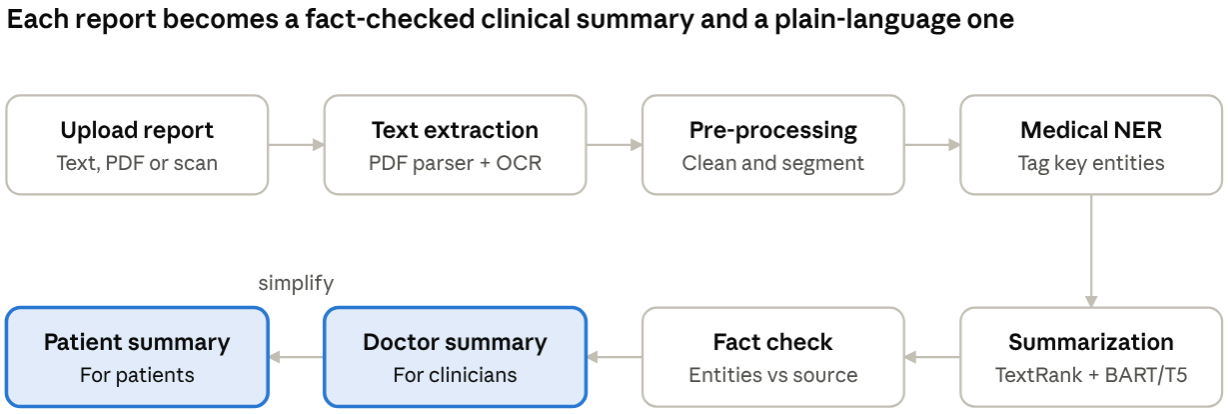
Current approach	How it works	Limitation
Manual reading by doctors	Clinician reads every page of the report	Slow; tiring across many patients; details can be missed
Hospital EHR systems	Store reports as documents or structured fields	Store data but do not summarize free-text notes
Generic summarizers (online tools)	Pick or rewrite sentences from any text	Not trained on medical language; may drop or distort clinical facts

Current approach	How it works	Limitation
Patient web searches	Patient looks up terms one by one	Unreliable sources; no context of the actual report

Drawbacks of the existing system: it is time-consuming, depends on the reader's expertise, gives no patient-friendly explanation, and cannot handle scanned reports or highlight abnormal values automatically.

6. Proposed System

The proposed system is an AI pipeline that reads a medical report and returns a fact-checked summary for doctors and a simplified one for patients. It combines OCR, clinical NLP and transformer-based summarization behind a simple web interface.



proposed system pipeline · 8 stages

The report flows through extraction, cleaning and entity tagging; the summary is checked against the source before it is shown.

How it improves on the existing system:

- **Saves time:** a multi-page report is reduced to a few key lines in seconds.
- **Two audiences:** a technical summary for clinicians and a jargon-free version for patients.
- **Handles any format:** works with typed text, PDFs and scanned or photographed reports.
- **Domain-aware:** uses models trained on biomedical text, so medical terms are understood correctly.
- **Highlights what matters:** abnormal lab values and critical findings are flagged.
- **Safer output:** entity-level fact checking reduces the risk of missing or invented information.

7. Methodology

The project follows a step-by-step NLP pipeline, from data collection to deployment.

1. **Data collection**
   - Public clinical text datasets such as MIMIC-III discharge summaries (access after credential training), MTSamples transcriptions, and PubMed abstracts for medical language.
   - Sample lab and radiology reports, with all personal details removed.
2. **Text extraction**
   - PDF text with PyMuPDF / pdfplumber.
   - Scanned images with Tesseract OCR, after OpenCV pre-processing (grayscale, thresholding, deskew).
3. **Pre-processing**
   - Cleaning: remove headers, footers, page numbers and extra whitespace.
   - Abbreviation expansion using a dictionary (e.g. "BP" to "blood pressure", "SOB" to "shortness of breath").
   - Sentence segmentation and tokenization with spaCy / scispaCy.
   - Section detection with rules and regular expressions (History, Findings, Impression, Medications).
4. **Medical entity recognition**
   - scispaCy and a BioBERT / ClinicalBERT token-classification model tag diseases, symptoms, drugs, dosages and tests.
   - Lab values are compared against standard reference ranges to flag abnormal results.
5. **Extractive summarization (baseline)**
   - TextRank: sentences become graph nodes, edges are weighted by cosine similarity of TF-IDF or sentence embeddings, and PageRank selects the top sentences.
   - Sentences that contain key medical entities get a higher score.
6. **Abstractive summarization (main model)**
   - Pre-trained transformer encoder-decoder models (BART, T5, or a biomedical variant) fine-tuned on report and summary pairs.
   - Long reports are split into chunks within the model's input limit, summarized, and then merged.
   - Beam search decoding with length and repetition penalties.
7. **Patient-friendly simplification**
   - A second pass rewrites the clinical summary in plain language, using a medical glossary to replace jargon.

8. **Fact checking**
   - Entities in the summary are matched against entities in the source report, so that no drug, value or diagnosis is invented or lost.
9. **Evaluation**
   - Automatic: ROUGE-1, ROUGE-2, ROUGE-L and BERTScore against reference summaries.
   - Manual: a small review of factual accuracy, completeness and readability.
10. **Deployment**
    - A Flask / FastAPI backend serves the model; a web frontend lets users upload a report and view or download the summary.

The core scoring step of TextRank is:

```
WS(V_i) = (1 - d) + d \sum_{V_j \in In(V_i)} \frac{w_{ji}}{\sum_{V_k \in Out(V_j)} w_{jk}} WS(V_j)
```

Here d is the damping factor (usually 0.85) and w is the similarity between two sentences.

8. Technologies Used

Category	Tools / Libraries
Programming language	Python 3.10+
NLP libraries	spaCy, scispaCy, NLTK
Deep learning	PyTorch, Hugging Face Transformers, Datasets
Pre-trained models	BART, T5, BioBERT / ClinicalBERT
Classical ML	scikit-learn (TF-IDF, cosine similarity), NetworkX (TextRank graph)
Text extraction / OCR	PyMuPDF, pdfplumber, Tesseract OCR (pytesseract), OpenCV
Evaluation	rouge-score, bert-score
Backend	Flask or FastAPI
Frontend	HTML, CSS, JavaScript (or Streamlit for a quick prototype)
Database	SQLite / MySQL for users, uploaded reports and summaries

Category	Tools / Libraries
Development tools	Jupyter Notebook, Google Colab (GPU for fine-tuning), VS Code, Git / GitHub
Hardware	PC with 8 GB+ RAM; NVIDIA GPU or Colab GPU for model training

9. Expected Outcomes

- A working web application that takes a medical report (text, PDF or scanned image) and returns a summary within a few seconds.
- Two summaries for each report: a short clinical summary for doctors and a plain-language summary for patients.
- Key medical entities (diagnoses, medicines, lab values) shown clearly, with abnormal values highlighted.
- A comparison of extractive (TextRank) and abstractive (BART / T5) methods, with the fine-tuned abstractive model expected to score higher on ROUGE and BERTScore.
- Summaries that keep the important clinical facts of the original report, checked by entity matching and manual review.
- Less time spent reading reports, and better understanding of their own health for patients.
- A base that can later be extended to multiple languages, voice output, or integration with hospital record systems.

10. References

1. A. Vaswani et al., "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
2. J. Devlin, M. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.
3. M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," *ACL*, 2020.
4. C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, 2020.
5. J. Lee et al., "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," *Bioinformatics*, 2020.
6. E. Alsentzer et al., "Publicly Available Clinical BERT Embeddings," *Clinical NLP Workshop*, 2019.

7. R. Mihalcea, P. Tarau, "TextRank: Bringing Order into Texts," *EMNLP*, 2004.
8. C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," *Text Summarization Branches Out, ACL Workshop*, 2004.
9. T. Zhang et al., "BERTScore: Evaluating Text Generation with BERT," *ICLR*, 2020.
10. M. Neumann et al., "ScispaCy: Fast and Robust Models for Biomedical Natural Language Processing," *BioNLP Workshop*, 2019.
11. A. Johnson et al., "MIMIC-III, a freely accessible critical care database," *Scientific Data*, 2016.
12. Hugging Face Transformers documentation: huggingface.co/docs/transformers
13. Tesseract OCR: github.com/tesseract-ocr/tesseract
14. MTSamples medical transcription samples: mtsamples.com

References are listed from memory; check page numbers and exact citation format against your college's style before submission.